Use the **Figshare Anolis dataset** as the primary source. It is a public dataset titled **“Analyses of the Anolis dataset”**.

1. Create folders

```bash
mkdir -p configs src scripts data/raw data/interim data/processed outputs/{logs,metrics,figures,models}
```

---

configs/dataset.yaml

```yaml
project:
  name: anolis-evolution-ml
  species_group: Anolis
```

```
  task:
classify_ecomorph_from_morphology

data:
  source_name: "Analyses of the Anolis
dataset"
  source_url: "https://figshare.com/
articles/dataset/
Analyses_of_the_Anolis_dataset/
4587259"
  download_url: "https://figshare.com/
ndownloader/articles/4587259/
versions/1"
  raw_dir: "data/raw"
  interim_dir: "data/interim"
  processed_dir: "data/processed"
  raw_zip: "data/raw/anolis_figshare.zip"
  extract_dir: "data/raw/anolis_figshare"
  manifest_csv: "data/interim/
manifest.csv"
  cleaned_csv: "data/processed/
anolis_cleaned.csv"

model:
  target_column: "ecomorph"
```

```
  test_size: 0.25
  random_state: 42
  classifier: "random_forest"

outputs:
  metrics_json: "outputs/metrics/
model_metrics.json"
  confusion_matrix_png: "outputs/
figures/confusion_matrix.png"
  model_path: "outputs/models/
random_forest_anolis.joblib"
```

---

src/ingest.py

python

```
from pathlib import Path
import argparse
import zipfile
import pandas as pd
```

```
import yaml


def load_config(path: str) -> dict:
    with open(path, "r") as f:
        return yaml.safe_load(f)


def build_manifest(extract_dir: Path,
manifest_csv: Path) -> None:
    files = []

    for path in extract_dir.rglob("*"):
        if path.is_file():
            files.append({
                "file_name": path.name,
                "relative_path": str(path),
                "suffix": path.suffix.lower(),
                "size_bytes": path.stat().st_size
            })

    manifest = pd.DataFrame(files)

manifest_csv.parent.mkdir(parents=True,
 exist_ok=True)
```

```
    manifest.to_csv(manifest_csv,
index=False)

    print(f"Wrote manifest:
{manifest_csv}")
    print(f"Files found: {len(manifest)}")


def extract_zip(raw_zip: Path,
extract_dir: Path) -> None:
    extract_dir.mkdir(parents=True,
exist_ok=True)

    with zipfile.ZipFile(raw_zip, "r") as z:
        z.extractall(extract_dir)

    print(f"Extracted: {raw_zip} ->
{extract_dir}")


def main(config_path: str) -> None:
    cfg = load_config(config_path)

    raw_zip = Path(cfg["data"]["raw_zip"])
    extract_dir = Path(cfg["data"]
```

```
["extract_dir"])
    manifest_csv = Path(cfg["data"]
["manifest_csv"])

    if not raw_zip.exists():
        raise FileNotFoundError(
            f"Missing raw zip: {raw_zip}\n"
            "Run scripts/download_data.sh
first."
        )

    extract_zip(raw_zip, extract_dir)
    build_manifest(extract_dir,
manifest_csv)


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--config",
default="configs/dataset.yaml")
    args = parser.parse_args()

    main(args.config)
```

---

src/clean.py

python

```
from pathlib import Path
import argparse
import pandas as pd
import yaml


def load_config(path: str) -> dict:
    with open(path, "r") as f:
        return yaml.safe_load(f)


def find_candidate_csv(extract_dir:
Path) -> Path:
    csv_files =
list(extract_dir.rglob("*.csv"))
```

```
    if not csv_files:
        raise FileNotFoundError(f"No CSV
files found in {extract_dir}")

    # Choose largest CSV as likely main
dataset
    csv_files = sorted(csv_files,
key=lambda p: p.stat().st_size,
reverse=True)
    return csv_files[0]


def clean_columns(df: pd.DataFrame) ->
pd.DataFrame:
    df = df.copy()
    df.columns = (
        df.columns
        .str.strip()
        .str.lower()
        .str.replace(" ", "_")
        .str.replace("-", "_")
        .str.replace(".", "_", regex=False)
    )
    return df
```

```
def infer_target_column(df:
pd.DataFrame, preferred: str) -> str:
    if preferred in df.columns:
        return preferred

    candidates = ["ecomorph", "ecotype",
"habitat", "island", "group"]

    for col in candidates:
        if col in df.columns:
            return col

    raise ValueError(
        f"Could not find target column.
Preferred target was '{preferred}'.\n"
        f"Available columns:
\n{list(df.columns)}"
    )


def main(config_path: str) -> None:
    cfg = load_config(config_path)

    extract_dir = Path(cfg["data"]
```

```
["extract_dir"])
    cleaned_csv = Path(cfg["data"]
["cleaned_csv"])
    target_preferred = cfg["model"]
["target_column"]

    input_csv =
find_candidate_csv(extract_dir)
    print(f"Using input CSV: {input_csv}")

    df = pd.read_csv(input_csv)
    df = clean_columns(df)

    target_col = infer_target_column(df,
target_preferred)

    df = df.dropna(axis=1, how="all")
    df = df.dropna(subset=[target_col])
    df = df.drop_duplicates()



cleaned_csv.parent.mkdir(parents=True,
exist_ok=True)
    df.to_csv(cleaned_csv, index=False)
```

```
    print(f"Wrote cleaned data:
{cleaned_csv}")
    print(f"Rows: {df.shape[0]}")
    print(f"Columns: {df.shape[1]}")
    print(f"Target column: {target_col}")
    print(f"Target classes:
{df[target_col].nunique()}")


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--config",
default="configs/dataset.yaml")
    args = parser.parse_args()

    main(args.config)
```

---

src/train.py

python

```
from pathlib import Path
import argparse
import json
import joblib
import pandas as pd
import yaml
import matplotlib.pyplot as plt

from sklearn.compose import
ColumnTransformer
from sklearn.ensemble import
RandomForestClassifier
from sklearn.impute import
SimpleImputer
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
)
from sklearn.model_selection import
```

```
train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import
OneHotEncoder, StandardScaler


def load_config(path: str) -> dict:
    with open(path, "r") as f:
        return yaml.safe_load(f)


def infer_target_column(df:
pd.DataFrame, preferred: str) -> str:
    if preferred in df.columns:
        return preferred

    candidates = ["ecomorph", "ecotype",
"habitat", "island", "group"]

    for col in candidates:
        if col in df.columns:
            return col

    raise ValueError(
        f"Target column not found.
```

```
Preferred target was '{preferred}'.\n"
        f"Available columns:
\n{list(df.columns)}"
    )


def main(config_path: str) -> None:
    cfg = load_config(config_path)

    cleaned_csv = Path(cfg["data"]
["cleaned_csv"])
    metrics_json = Path(cfg["outputs"]
["metrics_json"])
    cm_png = Path(cfg["outputs"]
["confusion_matrix_png"])
    model_path = Path(cfg["outputs"]
["model_path"])

    target_preferred = cfg["model"]
["target_column"]
    test_size = cfg["model"]["test_size"]
    random_state = cfg["model"]
["random_state"]

    df = pd.read_csv(cleaned_csv)
```

```
    target_col = infer_target_column(df,
target_preferred)

    y = df[target_col].astype(str)
    X = df.drop(columns=[target_col])

    # Keep usable predictors only
    X = X.drop(columns=[c for c in
X.columns if X[c].nunique(dropna=True)
<= 1])

    numeric_cols =
X.select_dtypes(include=["number"]).colu
mns.tolist()
    categorical_cols =
X.select_dtypes(exclude=["number"]).col
umns.tolist()

    if len(numeric_cols) +
len(categorical_cols) == 0:
        raise ValueError("No usable feature
columns found.")

    numeric_pipe = Pipeline([
        ("imputer",
```

```
SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])

    categorical_pipe = Pipeline([
        ("imputer",
SimpleImputer(strategy="most_frequent"
)),
        ("onehot",
OneHotEncoder(handle_unknown="ignor
e")),
    ])

    preprocessor = ColumnTransformer([
        ("numeric", numeric_pipe,
numeric_cols),
        ("categorical", categorical_pipe,
categorical_cols),
    ])

    model = RandomForestClassifier(
        n_estimators=300,
        random_state=random_state,
        class_weight="balanced",
    )
```

```
    pipeline = Pipeline([
        ("preprocess", preprocessor),
        ("model", model),
    ])

    X_train, X_test, y_train, y_test =
train_test_split(
        X,
        y,
        test_size=test_size,
        random_state=random_state,
        stratify=y if y.nunique() > 1 else
None,
    )

    pipeline.fit(X_train, y_train)
    preds = pipeline.predict(X_test)

    metrics = {
        "target_column": target_col,
        "n_rows": int(df.shape[0]),
        "n_features": int(X.shape[1]),
        "n_classes": int(y.nunique()),
        "classes": sorted(y.unique().tolist()),
```

```
        "accuracy":
float(accuracy_score(y_test, preds)),
        "macro_f1": float(f1_score(y_test,
preds, average="macro")),
        "weighted_f1": float(f1_score(y_test,
preds, average="weighted")),
        "classification_report":
classification_report(y_test, preds,
output_dict=True),
        "numeric_features": numeric_cols,
        "categorical_features":
categorical_cols,
    }


metrics_json.parent.mkdir(parents=True,
 exist_ok=True)
    with open(metrics_json, "w") as f:
        json.dump(metrics, f, indent=2)

    cm_png.parent.mkdir(parents=True,
exist_ok=True)
    cm = confusion_matrix(y_test, preds,
labels=sorted(y.unique()))
    disp =
```

```
ConfusionMatrixDisplay(confusion_matri
x=cm, display_labels=sorted(y.unique()))
    disp.plot(xticks_rotation=45)
    plt.tight_layout()
    plt.savefig(cm_png, dpi=300)
    plt.close()


model_path.parent.mkdir(parents=True,
exist_ok=True)
    joblib.dump(pipeline, model_path)

    print(f"Accuracy:
{metrics['accuracy']:.3f}")
    print(f"Macro F1:
{metrics['macro_f1']:.3f}")
    print(f"Wrote metrics: {metrics_json}")
    print(f"Wrote confusion matrix:
{cm_png}")
    print(f"Wrote model: {model_path}")


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--config",
```

```
default="configs/dataset.yaml")
    args = parser.parse_args()

    main(args.config)
```

---

scripts/download_data.sh

```bash
#!/usr/bin/env bash
set -euo pipefail

CONFIG="configs/dataset.yaml"
RAW_DIR="data/raw"
RAW_ZIP="data/raw/anolis_figshare.zip"
URL="https://figshare.com/
ndownloader/articles/4587259/
versions/1"

mkdir -p "$RAW_DIR"
```

```
echo "[1/1] Downloading Anolis Figshare
dataset..."
wget -O "$RAW_ZIP" "$URL"

echo "Saved to: $RAW_ZIP"
```

---

run_pipeline.sh

```bash
#!/usr/bin/env bash
set -euo pipefail

CONFIG="configs/dataset.yaml"
LOG_DIR="outputs/logs"
mkdir -p "$LOG_DIR"

echo "[1/4] Downloading data..."
bash scripts/download_data.sh 2>&1 |
```

```
tee "$LOG_DIR/01_download.log"

echo "[2/4] Ingesting data..."
python src/ingest.py --config "$CONFIG"
2>&1 | tee "$LOG_DIR/02_ingest.log"

echo "[3/4] Cleaning data..."
python src/clean.py --config "$CONFIG"
2>&1 | tee "$LOG_DIR/03_clean.log"

echo "[4/4] Training model...'
python src/train.py --config "$CONFIG"
2>&1 | tee "$LOG_DIR/04_train.log"

echo "Pipeline complete."
```

Install dependencies

bash

```
cat > requirements.txt <<'EOF'
pandas
pyyaml
scikit-learn
matplotlib
joblib
EOF

python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
chmod +x scripts/download_data.sh
run_pipeline.sh
./run_pipeline.sh
```

One note: the cleaning script is defensive because Figshare ZIPs often contain multiple files. It picks the largest CSV first, then looks for a likely target column such as ecomorph, ecotype, habitat, island, or group.